Computer through eyes

Table of Contents:

- **Problems :**

  - Many individuals (especially amputee) with physical disabilities face significant challenges in accessing and using computers, which are essential tools in today's digital age.
  - Often the touch pad of the laptop gets damaged and does not work properly.
  - Often the Computer's mouse doesn't work properly.

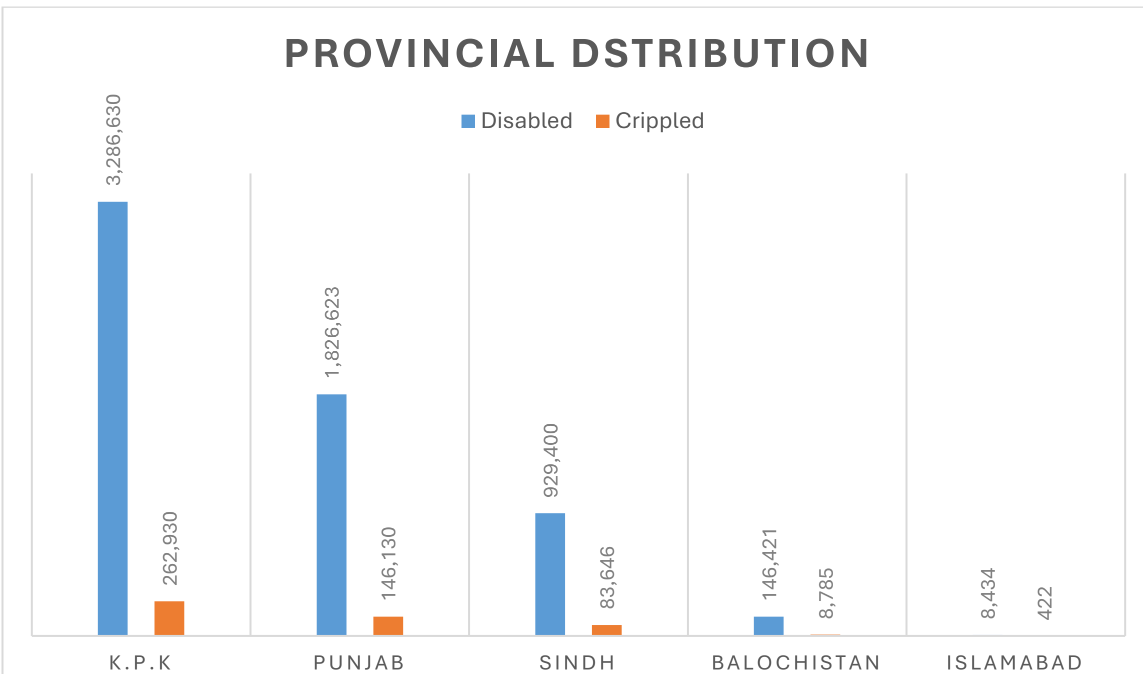
- **Targeted users**
  1. People with Disabilities (PWDs)
  2. People who can't get up from bed due to their illness. (bedridden)
  3. Computer / laptop users

- **Statistics :**

- **<u>According to W.H.O</u>** Report (7 March 2023) an estimated **1.3 billion** people experience significant disability. This represents **16%** of the world's population, or **1 in 6** of us.
- The 5th **<u>Population and Housing Census</u>** conducted in 1998 identified the population of Persons with Disabilities in Pakistan to be **2.38%** of the entire population. However as per the 6th Population and Housing Census of 2017, the percentage of **2%.**

- **<u>According to Pakistan Bureau of Statistics</u>**

  Total **3,286,630** are physically disabled in which **622,159** are crippled



- **<u>Proofs of Data</u>**

1. World Health Organization (W.H.O)

   Link : [World Health Organization (WHO)]

2. Population and Housing Census (P.H.C)

    Link : [Disability in Pakistan - Wikipedia]

3. Pakistan Bureau of Statistics (P.B.S)

    Link : [Pakistan Bureau of Statistics]

- **<u>Solution</u>**

I can develop a **computer software** for this purpose that track eyes movement to use computer.

Explanation :

Using **eye movement** to control the mouse **pointer** and **blinking** to make selections enables individuals with physical impairments to use computer easily. This software will directly runs on startup of computer, so that no third person would be required for anything.

Prompt for Picture : A man without arms is sitting and controlling his laptop using advanced retina-tracking technology.

Source code

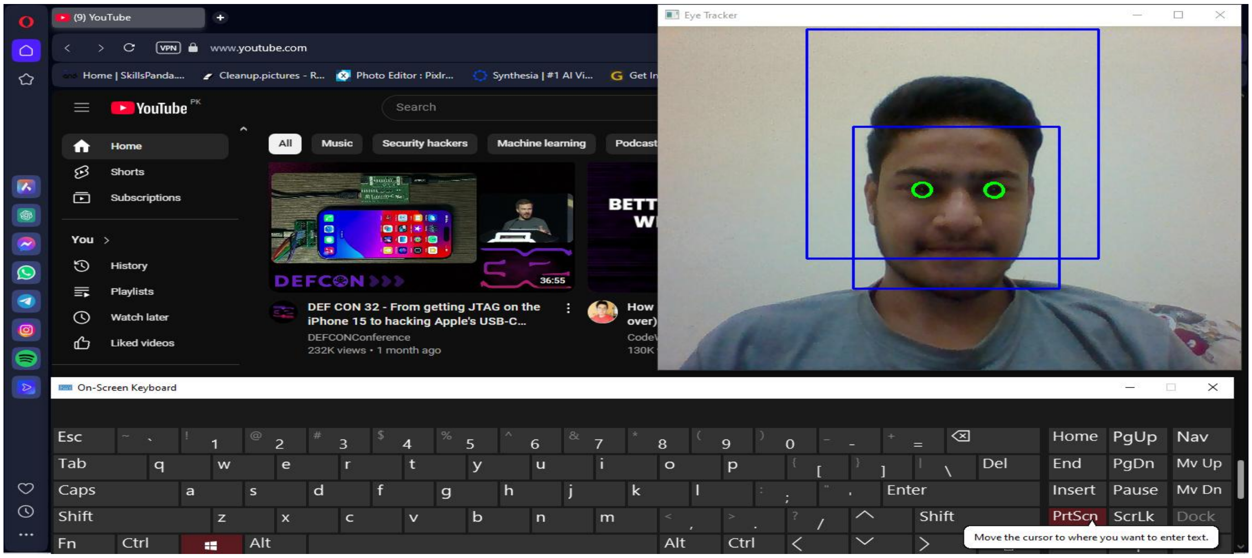
The code is written in **Python** programming language and in **VS-code** I.D.E.

```python
import cv2
import pyautogui
import time
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')
    blink_start_time = None
    exit_start_time = None
    screen_w, screen_h = pyautogui.size()
    eye_selected = None
    cap = cv2.VideoCapture(0)
    while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.flip(frame, 1)  # Flip frame horizontally
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(100, 100))
    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 0, 0), 2)
        roi_gray = gray[y:y+h, x:x+w]
        roi_color = frame[y:y+h, x:x+w]
        eyes = eye_cascade.detectMultiScale(roi_gray, scaleFactor=1.1, minNeighbors=10, minSize=(30, 30))
        for i, (ex, ey, ew, eh) in enumerate(eyes[:2]):
            eye_center_x = x + ex + ew // 2
            eye_center_y = y + ey + eh // 2
            cv2.circle(frame, (eye_center_x, eye_center_y), 10, (0, 255, 0), 2)
            if i == 0:
                eye_selected = 'left'
                mouse_x = int(screen_w / frame.shape[1] * eye_center_x)
                mouse_y = int(screen_h / frame.shape[0] * eye_center_y)
                pyautogui.moveTo(mouse_x, mouse_y)QQ
            elif i == 1:
                eye_selected = 'right'
        if len(eyes) == 0:
            if exit_start_time is None:
                exit_start_time = time.time()
            elif time.time() - exit_start_time >= 3:
                print("Exiting program...")
                cap.release()
                cv2.destroyAllWindows()
                exit()
        else:
            exit_start_time = None
        if len(eyes) == 1 and eye_selected == 'right':
            if blink_start_time is None:
                blink_start_time = time.time()
            elif time.time() - blink_start_time >= 1:
                pyautogui.click()
                print("Mouse clicked!")
                blink_start_time = None
        else:
            blink_start_time = None
    cv2.imshow('Eye Tracker', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

    cap.release()
    cv2.destroyAllWindows()
```

- ## Prototype Output

This is my screenshot of using the prototype of eye-tracking software.



- **<u>Control Instructions:</u>**
    - **Move Cursor**: Use your left eye.
    - **Click Mouse**: Blink with your right eye for 1 second.
    - **Exit Program**: Close both eyes for 3 sec or press "q" manually.

- **<u>Cost</u>**

This software will cost absolutly Rs.0 (Zero rupees) because it does not required any hardware resources.

- **<u>Time consumption</u>**

This software will required at least 6 months to be fully functional and available for end users.

- **<u>Credits :</u>**

1. **Youtube :** I have taken help from Youtube to install some helping softwares and libraries.
2. **Google :** I have taken help from google to search for statistical data.
3. **Chat G.P.T :** I have used chat G.P.T for debugging of code and picture.